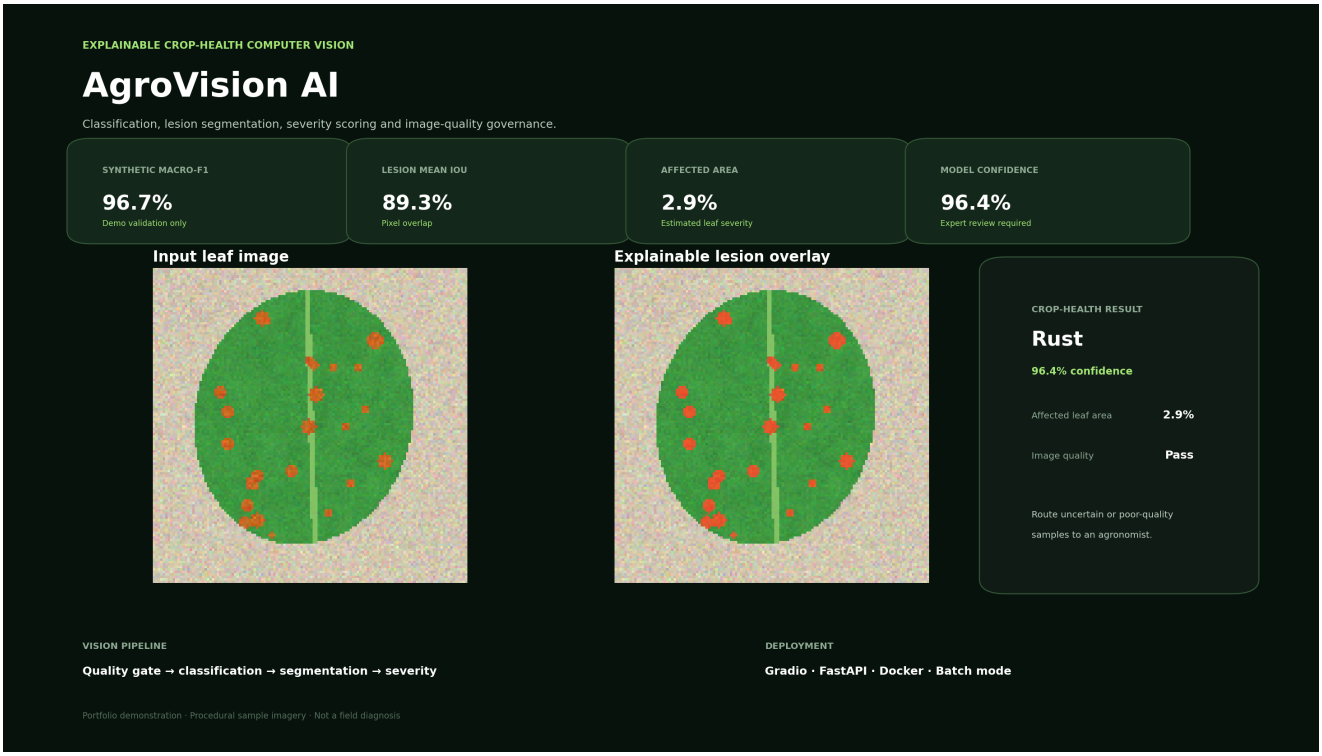
END-TO-END PROJECT STUDY GUIDE · DATA SCIENTIST INTERVIEW PREP

# AgroVision AI

Explainable crop-health computer vision: image-quality gating, leaf-disease classification, lesion segmentation, severity scoring and human-in-the-loop review




| What this guide covers | |
|---|---|
| Basics → proficiency | Business problem, architecture, every module of the code, the computer-vision and ML theory behind each step, metrics with formulas and worked examples |
| Verified by running the code | Every number in this guide was reproduced by re-running the project's own code (seed 42). Figures are generated from the real pipeline. |
| Honest critical review | Two significant issues found while verifying (leaf-mask bug and label-noise metric) with explanations, evidence and fixes. Know these before your interview. |
| Interview kit | Pitches, 40+ likely questions with model answers, glossary, formula sheet, numbers to memorise |

# Contents

**0. How to use this guide (tonight's plan) . . . 5**

**1. The project on one page . . . 6**

1.1 One-line definition . . . 6
1.2 The 30-second pitch . . . 6
1.3 The 2-minute version (add after the pitch) . . . 6
1.4 Key numbers . . . 6

**2. Problem, users and business value . . . 8**

2.1 The real-world problem . . . 8
2.2 Who uses it . . . 8
2.3 Why it's framed as decision *support*, not diagnosis . . . 8
2.4 How it creates business value . . . 8

**3. Tech stack and repository map . . . 9**

3.1 Libraries and why each was chosen . . . 9
3.2 Repository map . . . 9

**4. System architecture and request lifecycle . . . 11**

4.1 What happens when you click “Analyse” (AgroVisionService.analyse) . . . 11
4.2 The output contract (API response) . . . 12

**5. Data: the synthetic leaf generator . . . 13**

5.1 Why synthetic data? . . . 13
5.2 How generate_leaf() draws each image (128×128 RGB) . . . 13
5.3 Dataset construction in train_and_save() . . . 14

**6. Preprocessing and leaf segmentation . . . 15**

6.1 Preprocessing — as_rgb_array() . . . 15
6.2 Morphology fundamentals (you must know these) . . . 15
6.3 What segment_leaf() is supposed to do . . . 15
6.4 What it actually does — the leaf-mask bug . . . 16

**7. Lesion segmentation and severity . . . 18**

7.1 Colour rules — segment_lesions() . . . 18
7.2 Severity and overlay . . . 18
7.3 Segmentation quality per class (current code) . . . 18

**8. Feature engineering — the 42 features . . . 20**

8.1 Sobel gradient in one paragraph . . . 20

8.2 How strong is each feature on its own? . . . 20

**9. The model — Random Forest from first principles . . . 22**

9.1 Decision tree . . . 22
9.2 Random Forest = bagging + feature randomness . . . 22
9.3 Hyperparameters used . . . 22
9.4 Why a Random Forest (and not something else)? . . . 22
9.5 The saved bundle . . . 23

**10. Training procedure step by step . . . 24**

**11. Evaluation — metrics, formulas and results . . . 25**

11.1 Confusion matrix vocabulary (per class, one-vs-rest) . . . 25
11.2 Formulas . . . 25
11.3 Results . . . 25
11.4 Worked example — Healthy (left matrix) . . . 26
11.5 Why macro-F1 is the right headline metric here . . . 26

**12. Explainability . . . 27**

12.1 Three kinds of explanation in this project . . . 27
12.2 Two ways to measure feature importance . . . 27

**13. Image-quality gate . . . 28**

**14. Decision layer and human-in-the-loop . . . 29**

14.1 How to choose the threshold properly . . . 29

**15. Serving, deployment, testing and CI . . . 30**

15.1 Gradio web app (app.py) . . . 30
15.2 FastAPI (api.py) . . . 31
15.3 Docker, Makefile, CI . . . 32
15.4 Unit tests (tests/test_core.py) . . . 32

**16. Deep-learning path — MobileNetV3 transfer learning . . . 33**

16.1 Concepts . . . 33
16.2 Code walk-through . . . 33

**17. Critical review — what I found when re-running the code . . . 35**

**18. From demo to production — roadmap . . . 36**

18.1 Monitoring in one paragraph . . . 36

**19. Interview questions and model answers . . . 37**

Project overview . . . 37

Computer vision . . . 37
Machine learning . . . 38
Metrics & validation . . . 38
Deep learning . . . 39
Engineering & MLOps . . . 39
Business & ethics . . . 39

**20. Glossary . . . 40**

**21. One-page cheat sheet . . . 41**

**22. How to run everything . . . 42**

# 0. How to use this guide (tonight's plan)

This guide is ordered from fundamentals to expert-level detail. If you only have a few hours, follow this plan:

| Time | Do this | Sections |
|---|---|---|
| 30 min | Read the one-page summary and memorise the 30-second and 2-minute pitch | 1 |
| 60 min | Walk through the architecture and each pipeline stage; be able to draw the diagram from memory | 3 – 9 |
| 45 min | Metrics: precision, recall, F1, macro-F1, IoU, Dice — formulas + worked examples | 11 |
| 30 min | Critical review — the two big findings and how to talk about them confidently | 17 |
| 45 min | Interview Q&A — say the answers out loud | 19 |
| 15 min | Cheat sheet — numbers and formulas, last thing before sleeping / before the interview | 21 |

> **The golden rule for this interview**
>
> Interviewers for data-science roles value **judgement and honesty** more than a high score. This project's metrics come from **synthetic (procedurally generated) images**. Never claim it diagnoses real crops. Always say: *“The pipeline is complete and production-shaped; the metrics prove the engineering works, not field accuracy. Here is exactly how I would validate it on real data.”*
>
> Section 16 shows two issues I found when re-running the code. Raising them yourself (“when I audited my own pipeline I found…”) turns a potential weakness into the strongest part of your interview.

# 1. The project on one page

## 1.1 One-line definition

**AgroVision AI** takes a photo of a crop leaf and (1) checks if the photo is usable, (2) classifies the visible pattern as **Healthy, Leaf Spot, Rust or Blight**, (3) highlights the diseased pixels, (4) measures what percentage of the leaf is affected, and (5) routes uncertain cases to a human agronomist. It is served through a **Gradio** web app and a **FastAPI** REST API, packaged with **Docker**, tested with **unit tests + GitHub Actions CI**, and includes an optional **MobileNetV3 transfer-learning** script for real datasets.

## 1.2 The 30-second pitch

> **Say this**
>
> “AgroVision AI is an explainable crop-health computer-vision pipeline. It validates image quality, classifies four visible leaf patterns with a Random Forest on 42 engineered colour, texture and spatial features, segments lesion pixels to compute a severity percentage with a visual overlay, and routes low-confidence or poor-quality images to an expert. Because I didn't have labelled field data, I built a procedural image generator that gives exact ground-truth masks, so I could measure segmentation with IoU and Dice. It's served via Gradio and FastAPI in Docker with CI, and I wrote a MobileNetV3 transfer-learning path for real data.”

## 1.3 The 2-minute version (add after the pitch)

1. **Problem:** agronomists can't inspect every plant; farms need triage — which samples to look at first and how bad they are.
2. **Data:** 720 synthetic 128×128 leaf images (180 per class) with known lesion masks; ~3.5% labels deliberately flipped to mimic annotation noise.
3. **Two tracks:** an ML track answers *what* disease (Random Forest on 42 features) and a rule-based CV track answers *where and how much* (colour rules + morphology → lesion mask → severity %).
4. **Safety:** a quality gate (blur via Laplacian variance, brightness, leaf coverage) and a confidence threshold of 0.65 decide when a human must review.
5. **Results (synthetic):** macro-F1 0.967 against the stored labels, lesion IoU 0.893, Dice 0.941 — explicitly *not* field accuracy.
6. **Engineering:** service layer reused by UI, API and batch mode; Docker; 5 unit tests in CI; model card and governance tab.
7. **Next step:** real labelled data, grouped (farm/region/season) splits, CNN fine-tuning, calibration, OOD rejection, monitoring.

## 1.4 Key numbers

| Metric | Value | What it means |
|---|---|---|
| Images | 720 (540 train / 180 validation) | 180 per class, stratified 75/25 split, seed 42 |
| Features | 42 | colour stats, histograms, gradients, lesion ratios, quadrant ratios |
| Classifier | Random Forest, 420 trees | max_depth 16, min_samples_leaf 2, class_weight balanced_subsample |
| Macro-F1 / accuracy | 0.967 / 0.967 | vs. stored labels that include injected noise (vs. clean labels: 1.000 — see §17) |

| Metric | Value | What it means |
|---|---|---|
| Lesion IoU / Dice | 0.893 / 0.941 | pixel overlap of predicted vs. true lesion masks (135 diseased images) |
| Review threshold | confidence < 0.65 | or any image-quality warning → human review |
| Latency | ≈82 ms / image on CPU | full analyse() call; RF training ≈1 s |

# 2. Problem, users and business value

## 2.1 The real-world problem

Crop diseases such as leaf spot, rust and blight reduce yield, and early detection matters because many spread between plants. Expert agronomists are scarce and expensive; a farm or agri-business cannot have an expert look at every leaf. What they need is **triage**: a fast, consistent first pass that says *which samples look suspicious, how severe they are, and which need an expert first*.

## 2.2 Who uses it

| User | What they want from the system |
|---|---|
| Field scout / farmer | Take a phone photo, get immediate feedback, retake if the image is bad |
| Agronomist | A prioritised review queue with overlays showing *where* the problem is and a severity number |
| Quality-inspection team | Batch-process many images and get a consistent table of results |
| Engineering / product | An API to integrate with mobile apps, dashboards or ERP systems |

## 2.3 Why it's framed as decision *support*, not diagnosis

- Wrong actions are costly: unnecessary pesticide spraying, or missing a spreading blight.
- Visible symptoms are ambiguous: nutrient deficiencies, sunscald, pests and mixed infections can look similar.
- So the system **never acts alone**: every recommended action says “request agronomist confirmation”, and low-confidence / bad-quality images are explicitly flagged.
- The review queue also creates **new labelled data** — expert decisions can be fed back to retrain the model (human-in-the-loop / active learning).

## 2.4 How it creates business value

- **Prioritise scouting:** route expert time to the most severe / uncertain cases.
- **Standardise:** the same image always gets the same severity number — removes inspector-to-inspector variation.
- **Quantify:** severity % per leaf can be aggregated per field to track spread over time.
- **Auditability:** every decision stores probabilities, quality metrics and the overlay — useful for compliance and model monitoring.

# 3. Tech stack and repository map

## 3.1 Libraries and why each was chosen

| Library | Used for | Why |
|---|---|---|
| NumPy | image arrays, masks, maths | fast vectorised pixel operations |
| SciPy (ndimage) | morphology, Sobel, Laplacian, Gaussian filter, connected components | classical image processing without needing OpenCV |
| Pillow (PIL) | loading, resizing, overlays, contrast | standard Python image I/O |
| scikit-learn | RandomForest, train/test split, metrics, permutation importance | industry-standard classical ML |
| pandas | feature-importance CSV, batch result tables | tabular output |
| joblib | save/load the trained model bundle | efficient pickling of sklearn objects |
| Gradio + Plotly | interactive web UI + probability bar chart | fast ML demos; deployable to Hugging Face Spaces |
| FastAPI + Uvicorn | REST API (/health, /analyse) | async, type-checked, auto Swagger docs |
| Matplotlib | the demo poster image | static figures |
| PyTorch + torchvision (optional) | MobileNetV3 transfer learning | deep-learning path for real data |
| Docker, GitHub Actions | container + CI | reproducibility and automated testing |

## 3.2 Repository map

| Path | Role |
|---|---|
| src/config.py | Constants: paths, RANDOM_STATE=42, CLASSES, IMAGE_SIZE=128 |
| src/synthetic.py | generate_leaf(label, seed) → (image, true lesion mask, true leaf mask) |
| src/vision.py | as_rgb_array, segment_leaf, segment_lesions, extract_features, image_quality, create_overlay |
| src/modeling.py | FEATURE_NAMES, AgroVisionBundle (model + metrics + timestamp), train_and_save, load_bundle |
| src/service.py | AgroVisionService.analyse(image) — the full pipeline for one image; ACTIONS text per class |
| train.py | CLI entry: trains, saves artefacts, prints metrics |
| app.py | Gradio UI: single-image tab, batch tab, governance tab |
| api.py | FastAPI app: GET /health, POST /analyse |
| create_demo_preview.py | renders reports/agrovision_demo.png poster |
| training/train_transfer_learning.py | optional MobileNetV3-Small fine-tuning on an ImageFolder dataset |
| tests/test_core.py | 5 unit tests (generator, features, segmentation, service, model quality) |
| artifacts/ | agrovision_model.joblib, metrics.json, feature_importance.csv |
| data/samples/ | 12 sample PNGs (3 per class) used by the UI |
| docs/ | ARCHITECTURE.md, MODEL_CARD.md, INTERVIEW_GUIDE.md |
| Dockerfile, Makefile, .github/workflows/ci.yml | container, shortcuts (make train/test/demo/api), CI pipeline |

**Design pattern worth mentioning**

The **service layer** (`AgroVisionService`) holds all the logic once. The Gradio UI, the FastAPI endpoint, batch mode and the poster script all call the same `analyse()` — so there is one source of truth, no duplicated logic, and tests on the service cover every interface.

# 4. System architecture and request lifecycle

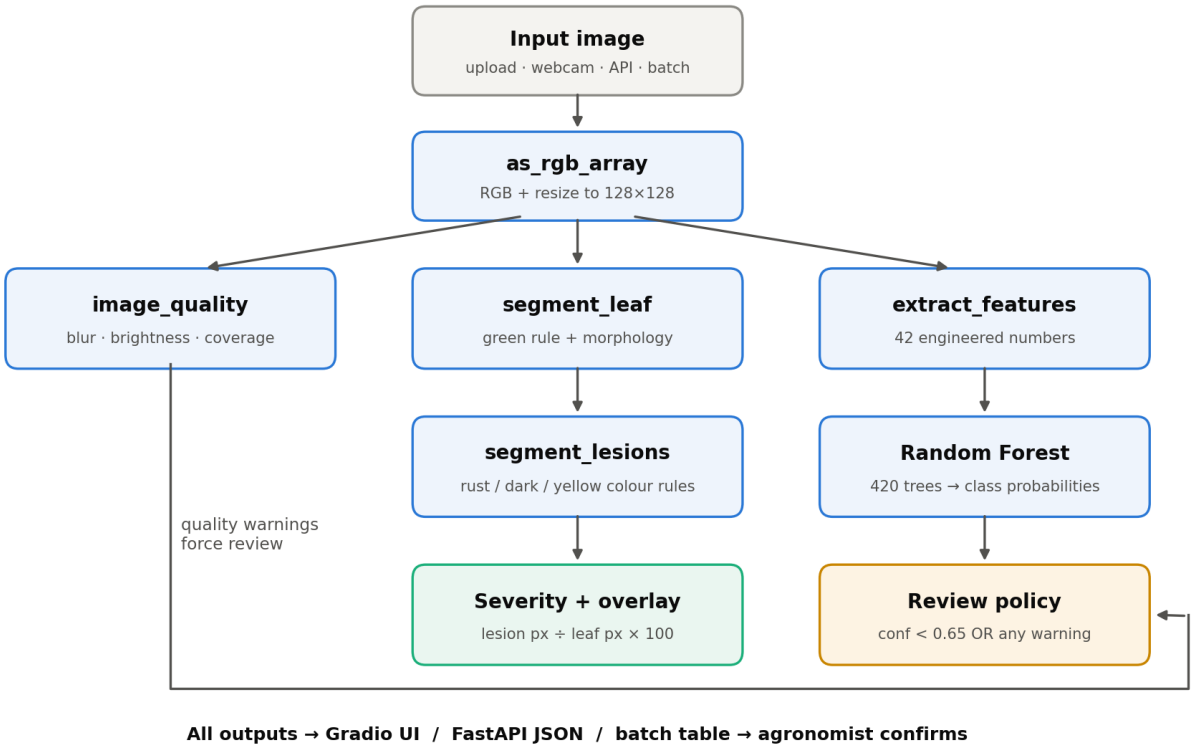



Figure 1. End-to-end pipeline. Two independent tracks run on the same 128×128 image: the ML track (right) decides WHAT the pattern is; the rule-based CV track (centre) decides WHERE and HOW MUCH. The quality gate (left) can force human review regardless of confidence.

## 4.1 What happens when you click “Analyse” (AgroVisionService.analyse)

1. **Validate input** — if no image, raise `ValueError`.
2. **as_rgb_array** — accepts a path, PIL image or NumPy array; converts to RGB; resizes to 128×128 (uint8).
3. **image_quality** — sharpness (Laplacian variance), brightness (mean grey), leaf coverage (%), list of warnings.
4. **segment_leaf** — boolean mask of leaf pixels (intended; see §6 for the bug).
5. **segment_lesions** — boolean mask of diseased pixels inside the leaf mask.
6. **bundle.predict** — extract 42 features → `predict_proba` → class with highest probability + confidence.
7. **severity** = lesion pixels ÷ leaf pixels × 100.
8. **needs_human_review** = confidence < 0.65 **or** any quality warning.
9. **Return** a dict: class, confidence, all probabilities, severity, pixel counts, quality, review flag, recommended action, overlay image, disclaimer.

## 4.2 The output contract (API response)

```
POST /analyse  (file=rust_1.png)  →  200 OK
{
  "predicted_class": "Rust",
  "confidence": 0.9642,
  "probabilities": {
    "Blight": 0.027,
    "Healthy": 0.0042,
    "Leaf Spot": 0.0046,
    "Rust": 0.9642
  },
  "severity_pct": 2.95,
  "leaf_area_pixels": 12984,
  "lesion_area_pixels": 383,
  "quality": {
    "sharpness": 1639.0,
    "brightness": 151.1,
    "leaf_coverage_pct": 79.2,
    "warnings": []
  },
  "needs_human_review": false,
  "recommended_action": "Check surrounding plants for orange pustules, reduce cross-p...",
  "disclaimer": "Portfolio decision-support demo. Results are not a..."
}
```

**Spot the clue**

`leaf_area_pixels = 12984` and `leaf_coverage_pct = 79.2`. 114 × 114 = 12 996 — the “leaf” is actually a 114×114 square. Every image returns ~79.2% coverage. This is the leaf-segmentation bug explained in §6 and §17.

# 5. Data: the synthetic leaf generator

## 5.1 Why synthetic data?

- **Reproducible:** anyone can run the whole project on a laptop with no dataset download; the same seed gives the same image.
- **Free pixel-perfect labels:** the generator returns the exact lesion mask, so segmentation can be measured with IoU/Dice — real datasets rarely have masks.
- **Controllable:** you can vary lighting, noise, lesion size and count deliberately.
- **Trade-off:** no real-world validity — real photos have clutter, shadows, multiple leaves, different crops, cameras and disease look-alikes.

## 5.2 How generate_leaf() draws each image (128×128 RGB)

1. **Background:** beige noise, colour ~ N([188,178,158], 12).
2. **Leaf:** a rotated ellipse (radii 30–37% and 38–45% of the image, angle ±0.38 rad); green colour ≈ (56,137,59) plus Gaussian-blurred texture noise.
3. **Vein:** a thin lighter-green line through the centre.
4. **Disease pattern** by class (table below); the true lesion mask is recorded at the same time.
5. **Field-like variation:** global brightness × U(0.83, 1.13) and sensor noise σ ∈ U(1, 5); clipped to 0–255.

| Class | Visual signature (generator) | What a model must notice |
|---|---|---|
| Healthy | plain green leaf + vein, no lesions | no lesions, uniform green |
| Leaf Spot | 5–12 brown discs, radius 3–7 px, each with a lighter tan ring | medium-size, dark, round spots |
| Rust | 18–41 tiny orange pustules, radius 1.3–3.2 px, spread everywhere | many small, red/orange, scattered |
| Blight | one large dark mottled patch near the leaf edge, with a yellow halo | large, dark, concentrated in one region |

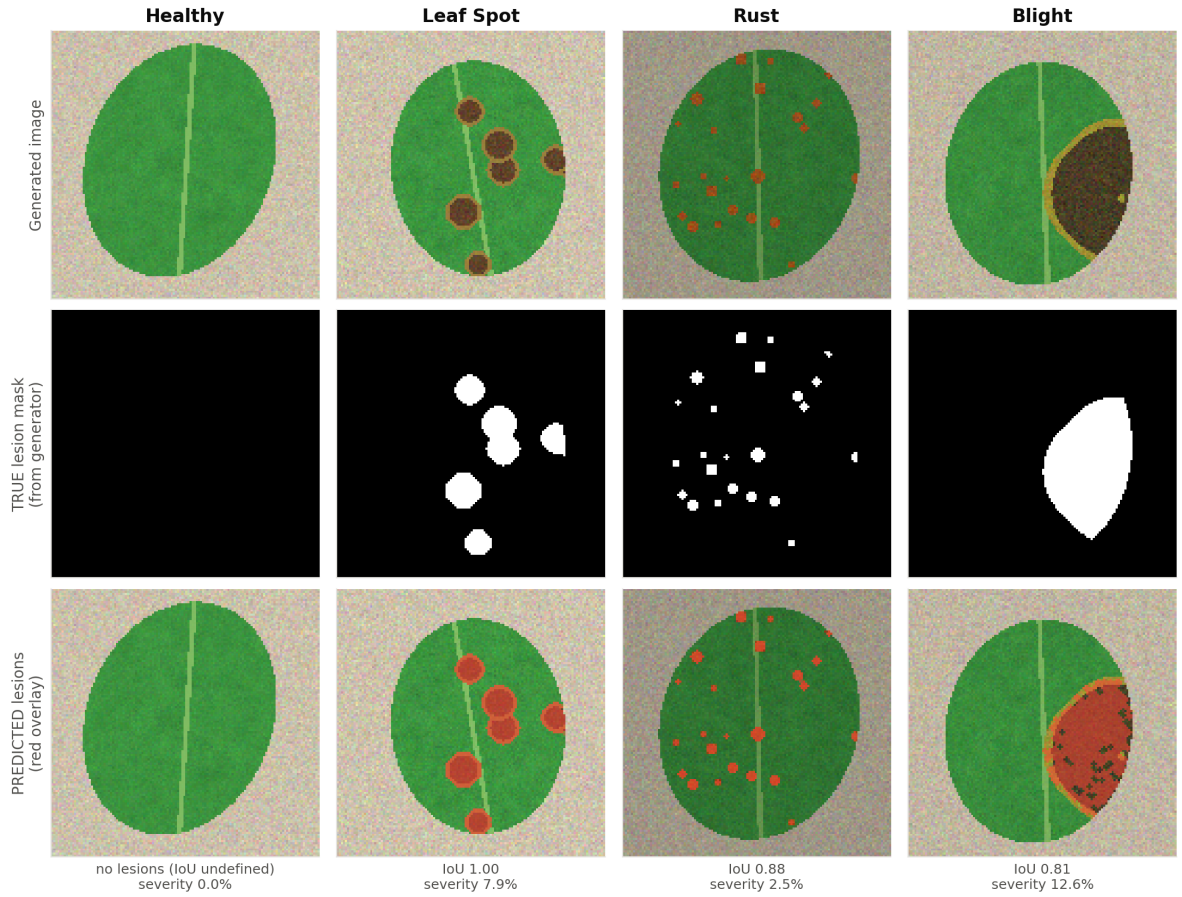



Figure 2. One generated example per class (top), its true lesion mask from the generator (middle) and the pipeline's predicted lesion overlay in red (bottom), with per-image IoU and the computed severity.

## 5.3 Dataset construction in train_and_save()

- 180 images per class × 4 = **720 images**, each with a deterministic seed: `42·100000 + class_index·180 + index`.
- **Injected label noise:** if `(index + class_index·7) % 29 == 0`, the label is replaced by the next class. This flips **25 of 720 labels (≈3.5%)** — Healthy→Leaf Spot, Leaf Spot→Rust, Rust→Blight, Blight→Healthy — to "simulate noisy field annotation".
- The first 3 images of each class are saved to `data/samples/` for the UI. Note: these are *training* images, and `healthy_1.png` is one of the flipped ones (index 0) — which is why the app shows it as Healthy with only 56% confidence (Leaf Spot 44%).
- For the first 45 images of each *diseased* class (135 images), predicted lesion masks are compared to the true masks to compute IoU and Dice.

# 6. Preprocessing and leaf segmentation

## 6.1 Preprocessing — as_rgb_array()

```
def as_rgb_array(image) -> np.ndarray:
    if isinstance(image, str):        image = Image.open(image)
    if isinstance(image, np.ndarray): image = Image.fromarray(image.astype(np.uint8))
    if not isinstance(image, Image.Image): raise ValueError(...)
    return np.asarray(image.convert("RGB").resize((IMAGE_SIZE, IMAGE_SIZE)), dtype=np.uint8)
```

- One function normalises every input type → a 128×128×3 uint8 array. Every other function calls it, so behaviour is consistent.
- **Limitation:** resizing to a square without preserving aspect ratio stretches non-square phone photos (a 4:3 photo is squashed). Better: resize the short side, then centre-crop or pad (letterbox).

## 6.2 Morphology fundamentals (you must know these)

| Operation | What it does to a binary mask | Typical use |
|---|---|---|
| Dilation | grows white regions by one pixel per iteration | join nearby fragments, fill small gaps |
| Erosion | shrinks white regions by one pixel per iteration | remove thin noise, undo dilation |
| Opening = erosion → dilation | removes small white specks, keeps big shapes | denoise a mask |
| Closing = dilation → erosion | fills small black holes/gaps, keeps outline | make blobs solid |
| Fill holes | any black region fully enclosed by white becomes white | solid object masks |
| Connected components (ndi.label) | gives each separate blob an ID; keep the largest | pick the main object |

## 6.3 What segment_leaf() is supposed to do

```
green_seed = (g > 45) & (g > b*1.06) & (g > r*.93) & ((g - np.minimum(r, b)) > 12)
mask = ndi.binary_dilation(green_seed, iterations=7)   # join fragments
mask = ndi.binary_fill_holes(mask)                     # absorb brown lesions inside the leaf
mask = ndi.binary_erosion(mask, iterations=7)          # shrink back
mask = ndi.binary_opening(mask, iterations=2)          # remove specks
keep the largest connected component
```

The idea is sensible: find “green-dominant” pixels, then use dilation + hole filling so that brown/orange lesions *inside* the leaf are included, and keep the biggest blob.

## 6.4 What it actually does — the leaf-mask bug

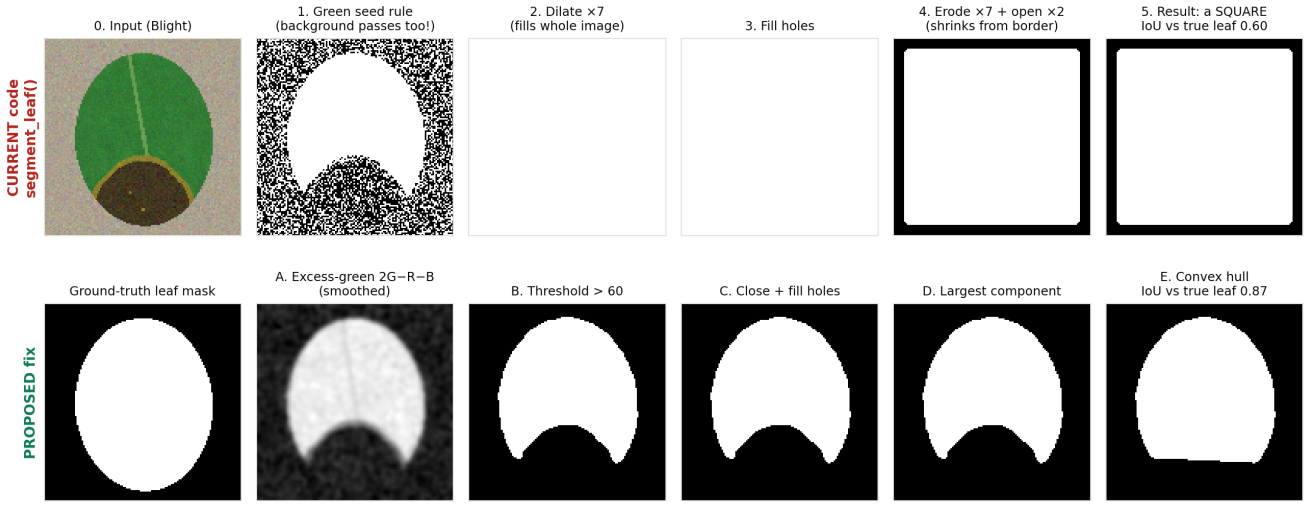



Figure 3. Top: the current segment_leaf() step by step. The beige background passes the "green" rule on about half its pixels, dilation ×7 fills the whole image, and erosion shrinks only from the border — the result is the same 114×114 square for every image. Bottom: a proposed fix using the smoothed excess-green index (2G − R − B), threshold, closing, largest component and convex hull.

**Root cause:** the background mean colour (188, 178, 158) *satisfies* the green rule: 178 > 158×1.06 = 167.5; 178 > 188×0.93 = 174.8; 178 − 158 = 20 > 12. With noise, about half the background pixels pass. Seven dilations then merge everything into one full-image blob; erosion with the default border value eats 7 px from the image edges → a square of $(128 - 14)^2 = 12\,996$ pixels = 79.2% of the image.

| Class (40 images each) | Leaf-mask IoU: current | Leaf-mask IoU: fix | True severity | Severity: current | Severity: fix |
|---|---|---|---|---|---|
| Healthy | 0.543 | 0.977 | 0.0% | 0.0% | 0.0% |
| Leaf Spot | 0.543 | 0.977 | 15.0% | 8.2% | 14.5% |
| Rust | 0.543 | 0.975 | 6.3% | 3.1% | 5.5% |
| Blight | 0.543 | 0.900 | 26.4% | 11.4% | 14.7% |

**Consequences:**

- **Severity is under-estimated by roughly half**, because the denominator is the square (background included), not the leaf.
- The **leaf-coverage quality check never fires** on normal images (always 79.2%), so a tiny leaf in a big frame is not flagged (see Figure 8).
- The `leaf_coverage` feature is a constant → useless; colour features are computed over leaf *and* background pixels.
- Classification still works (the lesion features still separate the classes) and lesion IoU is unaffected because the background never matches the lesion colour rules.

> **How to talk about it**
>
> "When I audited the pipeline I noticed leaf coverage was always 79.2%. I traced it to the green threshold: the beige background passes it, and the dilation step floods the image. Switching to a smoothed excess-green index with a convex hull raises leaf-mask IoU from 0.54 to ~0.95 and makes severity much closer to ground truth. It taught me to unit-test intermediate outputs, not just the final metric."

```
# Proposed fix (tested): excess-green + convex hull
exg  = ndi.gaussian_filter(2*g - r - b, 1.5)        # background ≈ 10, leaf ≈ 159
mask = ndi.binary_fill_holes(ndi.binary_closing(exg > 60, iterations=4))
mask = largest_connected_component(mask)
mask = convex_hull(mask)   # leaves are roughly convex → recovers lesions on the edge
```

# 7. Lesion segmentation and severity

## 7.1 Colour rules — segment_lesions()

| Rule | Condition (per pixel, inside leaf mask) | Targets |
|---|---|---|
| rust | R > 1.25·G and R > 1.65·B and R > 105 | orange/red rust pustules |
| dark | mean(R,G,B) < 93 and R ≥ 0.92·G | brown/necrotic tissue (spots, blight core) |
| yellow | R > 105 and G > 88 and B < 85 and R > 1.02·G | yellow/tan halos and rings (chlorosis) |

Lesion mask = leaf ∧ (rust ∨ dark ∨ yellow), then **opening** (remove isolated noisy pixels) and **closing** (fill tiny gaps).

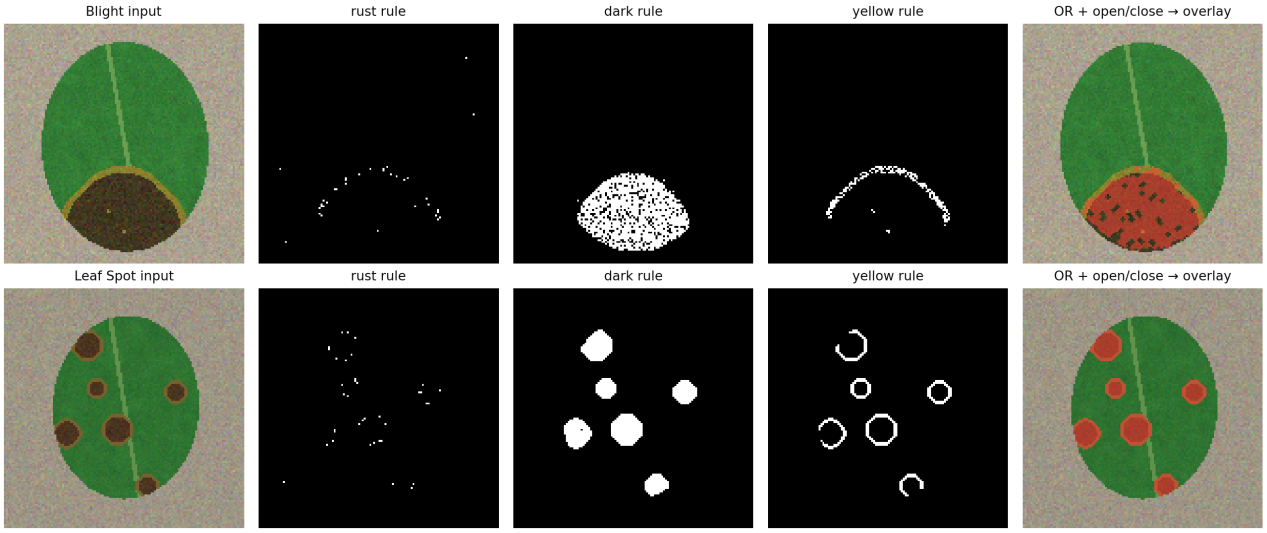



Figure 4. What each colour rule picks up on a Blight leaf (top) and a Leaf Spot leaf (bottom), and the combined overlay. Notice the speckled “dark” response inside the blight patch — noise pushes some pixels above the intensity threshold, which is why Blight has the lowest lesion IoU.

## 7.2 Severity and overlay

- **Severity %** = lesion pixels ÷ leaf pixels × 100 — an area-based estimate of how much of the leaf is affected (agronomists use similar % leaf area scales).
- **Overlay:** lesion pixels are painted red (RGBA 255,70,55, alpha 135) and alpha-composited on the image — a direct visual explanation of *where* the model sees disease.
- **Why rule-based?** No training data needed, fully transparent, fast. **Why it won't survive real data:** fixed RGB thresholds break under different lighting, white balance, crops and look-alike colours (soil, dead leaves). Production → learned segmentation (U-Net, DeepLab, or SAM2 fine-tuned on expert masks).

## 7.3 Segmentation quality per class (current code)

| Class | Lesion IoU (40 imgs) | Comment |
|---|---|---|
| Leaf Spot | 0.994 | large, high-contrast brown discs + rings → near perfect |
| Rust | 0.880 | tiny pustules: a 1-pixel boundary error is a big % of a small object |
| Blight | 0.795 | noisy mottled core: some pixels miss the 'dark' threshold |
| Mean (reported, 135 imgs) | 0.893 (Dice 0.941) | Healthy excluded — no true lesions, IoU undefined |

**Caveat you should mention**

The thresholds were hand-tuned on images from the *same generator* that produces the ground truth — so the IoU is optimistic (circular evaluation). On real images, masks must be drawn by agronomists and evaluated on a held-out set.

# 8. Feature engineering — the 42 features

A Random Forest can't look at raw pixels the way a CNN does, so domain knowledge is encoded as numbers. Most statistics are computed only over pixels inside the leaf mask (which, because of the bug, also contains background).

| # | Group | Names | Count | Why it helps |
|---|---|---|---|---|
| 1 | Colour mean | mean_r, mean_g, mean_b | 3 | disease shifts colour away from green |
| 2 | Colour spread | std_r, std_g, std_b | 3 | lesions make colour patchy → higher std |
| 3 | Colour histograms | hist_{r,g,b}_{0..7} (8 bins each, density) | 24 | distribution, e.g. how many very-low-green pixels |
| 4 | Texture / edges | gradient_mean, gradient_std, gradient_p90 (Sobel magnitude) | 3 | spots create sharp edges; rust = many small edges |
| 5 | Coverage & colour ratios | leaf_coverage, lesion_ratio, red_ratio, dark_ratio, yellow_ratio | 5 | direct domain signals: red=rust, dark=spot/blight, yellow=blight halo |
| 6 | Spatial | lesion_quadrant_0..3 (lesion fraction per image quadrant) | 4 | blight is concentrated in one quadrant; rust is spread evenly |
| | **Total** | | **42** | |

## 8.1 Sobel gradient in one paragraph

Sobel filters approximate the image derivative in x and y. The gradient magnitude $\sqrt{(gx^2 + gy^2)}$ is large at edges (sudden brightness changes) and small in flat regions. Mean, std and the 90th percentile of this magnitude summarise “how much edge/texture” the leaf has — lesions add edges.

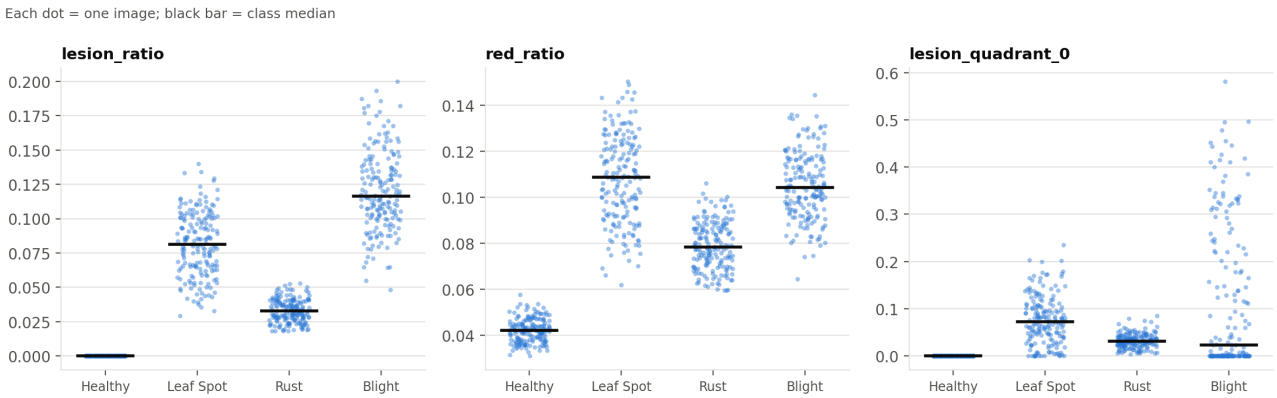



Figure 5. Three engineered features across all 720 images (true classes). lesion_ratio alone separates the four classes well; the quadrant feature shows Blight's concentration (high in some quadrants, zero in others). Healthy's non-zero red_ratio comes from background pixels inside the square “leaf” mask.

## 8.2 How strong is each feature on its own?

| Feature alone (depth-3 tree) | Validation accuracy |
|---|---|
| lesion_ratio | 86.1% |
| red_ratio | 71.7% |
| yellow_ratio | 56.1% |
| dark_ratio | 25.6% |
| gradient_mean | 31.7% |

| Feature alone (depth-3 tree) | Validation accuracy |
|---|---|
| mean_g | 31.7% |
| All 42 (Random Forest) | 100% vs. true labels |

Chance level is 25%. `lesion_ratio` alone reaches 86% — the synthetic task is easy because the generator draws very distinct patterns.

# 9. The model — Random Forest from first principles

## 9.1 Decision tree

A decision tree asks a sequence of yes/no questions on features (e.g. “lesion_ratio > 0.05?”). At each node it picks the feature and threshold that best separate the classes, measured by **Gini impurity**: Gini = $1 - \Sigma\ p_k^2$, where $p_k$ is the fraction of class k in the node. A pure node has Gini 0. Trees are interpretable but high-variance: small data changes produce very different trees and they overfit.

## 9.2 Random Forest = bagging + feature randomness

- **Bootstrap (bagging):** each tree trains on a random sample of rows drawn with replacement.
- **Feature subsampling:** at each split only a random subset of features (default $\sqrt{42} \approx 6$) is considered → trees are decorrelated.
- **Aggregation:** `predict_proba` = average of the trees' class-probability estimates; prediction = arg-max.
- **Effect:** averaging many decorrelated high-variance trees reduces variance (overfitting) while keeping low bias.

## 9.3 Hyperparameters used

| Parameter | Value | Meaning / why |
|---|---|---|
| n_estimators | 420 | number of trees; more = more stable probabilities, diminishing returns |
| max_depth | 16 | caps tree depth → regularisation |
| min_samples_leaf | 2 | a leaf must hold ≥ 2 samples → smoother probabilities, less overfitting |
| class_weight | balanced_subsample | re-weights classes inversely to frequency within each bootstrap sample (important for real, imbalanced data) |
| random_state | 42 | reproducibility |
| n_jobs | -1 | use all CPU cores |

Hyperparameters were set manually, not tuned. In an interview, say you would tune with cross-validated randomised search, scored on macro-F1, with grouped folds on real data.

## 9.4 Why a Random Forest (and not something else)?

| Option | Pros | Cons here |
|---|---|---|
| **Random Forest (chosen)** | great on small tabular feature sets, no scaling, robust, fast on CPU, gives probabilities and importances | relies on hand-made features; probabilities not calibrated |
| Logistic regression | simple, calibrated-ish, interpretable coefficients | linear boundaries; needs scaling |
| SVM | strong on small data | no native probabilities; needs scaling and tuning |
| XGBoost / LightGBM | often the best on tabular data | more tuning; overkill for 720 easy samples |
| CNN (e.g. MobileNetV3) | learns features from pixels; best for real photos | needs thousands of labelled images and a GPU ideally |

## 9.5 The saved bundle

```
@dataclass
class AgroVisionBundle:
    classifier: RandomForestClassifier
    metrics: dict
    trained_at: str          # ISO timestamp → exposed by GET /health (model versioning)
    def predict(self, image) -> dict:
        features = extract_features(image).reshape(1, -1)
        probabilities = self.classifier.predict_proba(features)[0]
        ...  # returns predicted_class, confidence (max prob), probabilities
```

Saved with joblib to `artifacts/agrovision_model.joblib`. `load_bundle()` trains automatically if the file is missing. Packaging metrics + timestamp with the model is a lightweight form of model versioning.

# 10. Training procedure step by step

1. Create artefact and sample folders.
2. For each class and index: generate the image (seeded) → extract 42 features → assign label (with deterministic noise) → save first 3 as samples → for the first 45 diseased images compute lesion IoU and Dice against the true mask.
3. **Stratified split** 75/25 (`train_test_split(..., stratify=y, random_state=42)`) → 540 train, 180 validation; class proportions preserved.
4. Fit the Random Forest on the training features.
5. Predict the validation set; compute accuracy, macro-F1, per-class F1, the confusion matrix and a classification report.
6. Compute permutation importance on the validation set (5 repeats, scoring macro-F1) → feature_importance.csv.
7. Save the bundle (joblib) and metrics.json (with an explicit `data_scope` note: “procedurally generated… not field-validated”).

```
$ python train.py
Training images: 540
Validation images: 180
Synthetic macro-F1: 0.967
Lesion mean IoU: 0.893
Important: synthetic validation does not represent field accuracy.
```

**Why stratify?**

Stratification keeps the class proportions the same in train and validation. With small datasets a random split can by chance put too few samples of one class in validation, making the per-class metrics unstable.

**Leakage — know this cold**

For synthetic data a random split is fine (every image is independent). For real data it is **wrong**: photos of the same plant, field, day or camera end up in both train and test and inflate scores. Use **grouped splits** (GroupKFold by farm / region / season / plant ID) so the test set looks like truly new deployments.

# 11. Evaluation — metrics, formulas and results

## 11.1 Confusion matrix vocabulary (per class, one-vs-rest)

| Term | Meaning (for class “Blight”) |
|---|---|
| TP | image is Blight and model says Blight |
| FP | model says Blight but it isn't (false alarm) |
| FN | image is Blight but model says something else (missed disease) |
| TN | not Blight, and model didn't say Blight |

## 11.2 Formulas

| Metric | Formula | Plain English |
|---|---|---|
| Accuracy | (correct predictions) / (all predictions) | misleading if classes are imbalanced |
| Precision | TP / (TP + FP) | when it says Blight, how often is it right? |
| Recall (sensitivity) | TP / (TP + FN) | of all real Blight, how much did it catch? |
| F1 | 2 · P · R / (P + R) | harmonic mean — high only if both P and R are high |
| Macro-F1 | mean of per-class F1 (equal weight) | rare classes count as much as common ones |
| Weighted-F1 | per-class F1 weighted by support | dominated by frequent classes |
| Micro-F1 | F1 from pooled TP/FP/FN | = accuracy for single-label multiclass |
| IoU (Jaccard) | \|A ∩ B\| / \|A ∪ B\| | pixel overlap ÷ union of predicted and true masks |
| Dice (F1 for pixels) | 2\|A ∩ B\| / (\|A\| + \|B\|) | always ≥ IoU; Dice = 2·IoU / (1 + IoU) |

## 11.3 Results

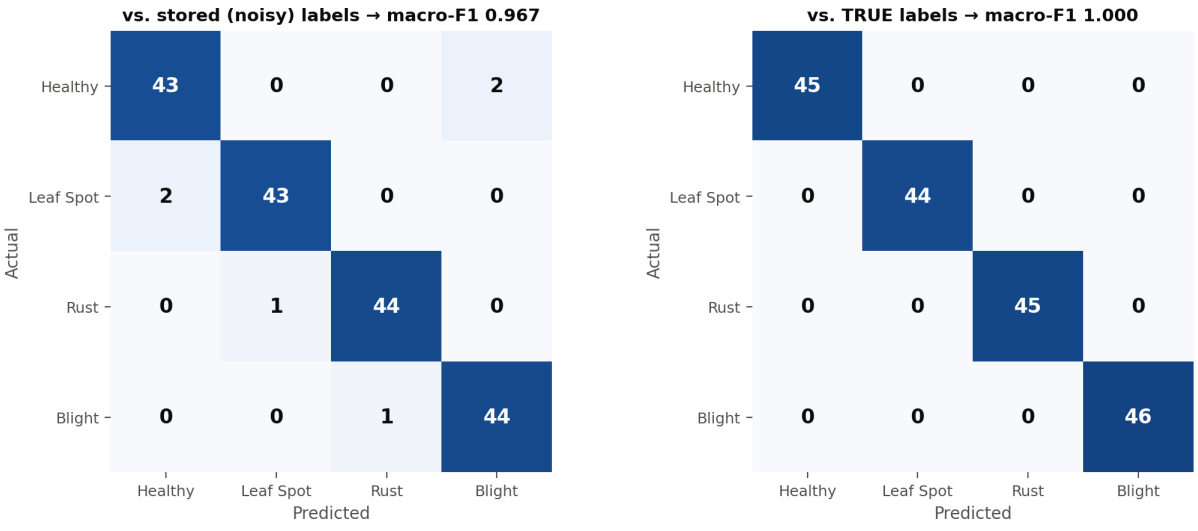



Figure 6. Validation confusion matrices (180 images). Left: against the stored labels, which include the injected noise — this is where the reported 0.967 comes from. Right: the same predictions against the generator's true labels — zero errors.

| Class | Precision | Recall | F1 | Support |
|---|---|---|---|---|
| Healthy | 0.956 | 0.956 | 0.956 | 45 |
| Leaf Spot | 0.977 | 0.956 | 0.966 | 45 |
| Rust | 0.978 | 0.978 | 0.978 | 45 |
| Blight | 0.957 | 0.978 | 0.967 | 45 |
| **Macro average** | 0.967 | 0.967 | **0.967** | 180 |

## 11.4 Worked example — Healthy (left matrix)

Row “Healthy” (actual): 43 predicted Healthy, 2 predicted Blight → TP = 43, FN = 2. Column “Healthy” (predicted): 43 correct + 2 from the Leaf Spot row → FP = 2. Precision = 43/45 = 0.956; Recall = 43/45 = 0.956; F1 = 0.956. Do the same for each class and average → macro-F1 0.967.

## 11.5 Why macro-F1 is the right headline metric here

- In real fields most leaves are healthy (say 90%). A model that always says “Healthy” scores 90% accuracy and is useless. Macro-F1 exposes that because each disease class's F1 would be 0.
- For disease classes, **recall** matters most: a missed blight (FN) costs more than an extra expert check (FP). In production you'd report per-class recall and choose thresholds per class.
- For segmentation, IoU and Dice are standard; IoU is stricter. Report both plus severity error (|predicted % − true %|), which is what users actually see.

> **Interview-grade insight**
>
> All 6 “errors” are images whose stored label was flipped. The model predicted their *true* class with high confidence (0.89, 0.93, 0.94, 0.96, 0.98, 0.99). Confident disagreement with a label is exactly how label-noise detection tools (e.g. cleanlab / confident learning) find annotation errors in real datasets.

# 12. Explainability

## 12.1 Three kinds of explanation in this project

- **Visual (local):** the red lesion overlay shows exactly which pixels drove the severity estimate — understandable by a non-technical agronomist.
- **Probabilistic (local):** the bar chart of all four class probabilities shows how confident and how "torn" the model is.
- **Feature importance (global):** which features the model relies on overall.

## 12.2 Two ways to measure feature importance

| Method | How it works | Pitfalls |
|---|---|---|
| MDI (impurity) — feature_importances_ | sum of Gini decrease contributed by each feature across all tree splits | computed on training data; biased towards high-cardinality / continuous features |
| Permutation importance (used in project) | shuffle one feature on validation data; importance = drop in score | correlated features share credit → each looks unimportant |

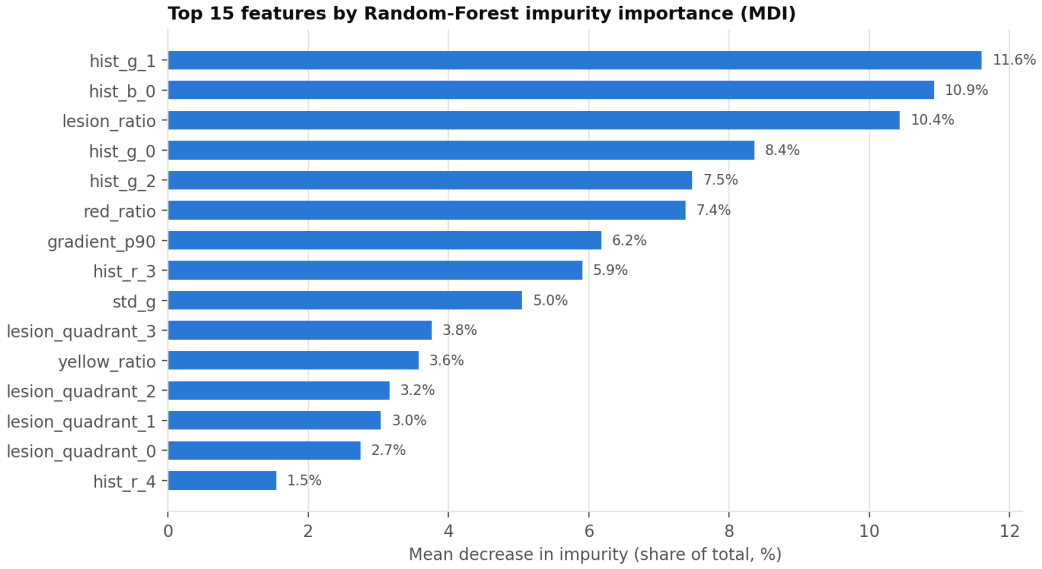



Figure 7. Impurity-based (MDI) importance from the reproduced model. Colour histograms of the green and blue channels and lesion_ratio dominate.

**Why feature_importance.csv is almost all zeros**

In the project's permutation-importance file, 40 of 42 features score exactly 0. Two reasons: (1) the features are heavily redundant — 41 feature pairs have |correlation| > 0.9 — so when one is shuffled the forest uses its correlated twin and the score doesn't drop; (2) the model is perfect on clean data, so there is little performance to lose.

Fixes: grouped permutation importance (shuffle a whole feature group together), SHAP values (TreeSHAP is exact and fast for forests), or drop redundant features first.

# 13. Image-quality gate

Garbage in, garbage out: a blurry or dark photo can still produce a confident-looking prediction. The quality gate catches bad inputs *before* anyone trusts the output.

| Check | Method | Rule | Why |
|---|---|---|---|
| Blur | variance of the Laplacian of the grey image | < 65 → “image may be blurred” | the Laplacian (2nd derivative) responds to edges; sharp image → many strong responses → high variance |
| Too dark | mean grey level | < 55 | colour rules and features become unreliable |
| Overexposed | mean grey level | > 220 | colours saturate to white |
| Framing | leaf mask % of image | < 12% → “leaf occupies too little” | too few leaf pixels to judge (currently never fires — §6) |




Figure 8. The quality gate on modified versions of one rust leaf. Blur is caught (sharpness 1) and correctly forces review even though the model wrongly says Healthy. Dark is caught. The overexposed image (brightness 217) slips just under the 220 threshold but is flagged by low confidence. The small-leaf case is NOT caught: coverage still reports 79% because of the leaf-mask bug.

**Improvements:** thresholds should be calibrated on real phone photos (they depend on resolution — measure blur *before* downsizing); add glare/specular-highlight detection, colour-cast (white balance) checks, and an out-of-distribution check for “this isn't a leaf”.

# 14. Decision layer and human-in-the-loop

```
needs_review = prediction["confidence"] < .65 or bool(quality["warnings"])
```

- **Confidence** = the highest class probability = fraction of tree votes (averaged leaf probabilities) for the winning class.
- **Two independent triggers:** the model is unsure, or the input is unreliable. Either sends the case to an agronomist.
- **Recommended action** per class (ACTIONS dict) — all are triage steps and all require agronomist confirmation; none recommends a chemical treatment.

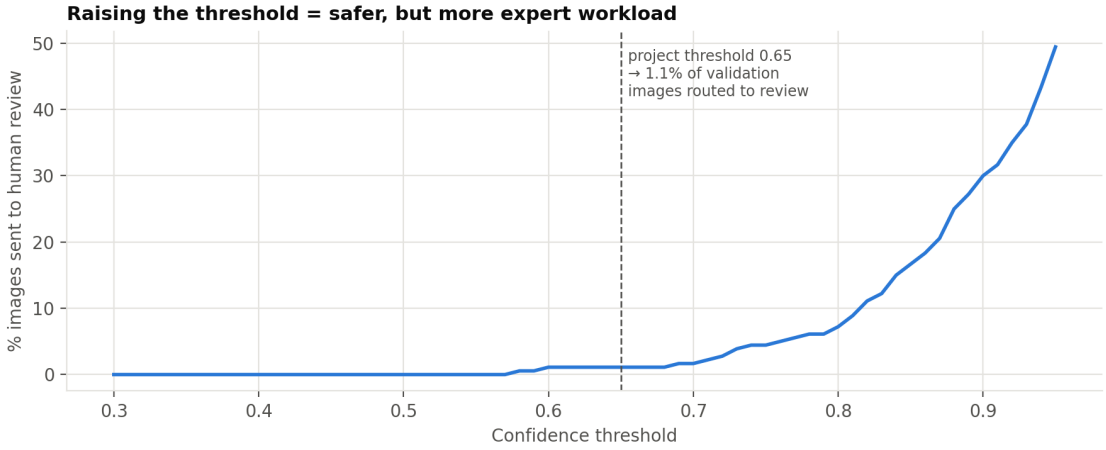



Figure 9. Workload trade-off on the validation set: the share of images routed to human review as a function of the confidence threshold. At 0.65 only 1.1% are routed; at 0.9, about 30%.

## 14.1 How to choose the threshold properly

1. **Calibrate first** — Random Forest probabilities are not true probabilities (they tend to be pulled away from 0 and 1). Use CalibratedClassifierCV (isotonic or Platt/sigmoid) and check a reliability diagram / Expected Calibration Error (ECE).
2. **Use costs:** define the cost of a missed disease vs. the cost of an expert review; pick the threshold that minimises expected cost on a validation set.
3. **Per-class thresholds:** a stricter threshold for Blight (urgent, spreads fast) than for Healthy.
4. **Monitor** the review rate and expert-override rate in production; a rising override rate signals drift.

> **Selective prediction**
>
> This pattern — predict only when confident, otherwise abstain/defer to a human — is called **selective classification** or **learning to defer**. You can plot a risk–coverage curve: error rate vs. fraction of cases the model handles automatically.

# 15. Serving, deployment, testing and CI

## 15.1 Gradio web app (app.py)

- **Hero banner** with KPI cards (macro-F1, IoU, number of classes, decision policy) read from the saved metrics.
- **Tab 1 – Single leaf analysis:** upload / webcam / clipboard → overlay image, Markdown summary (class, confidence, severity, review, next step, disclaimer), Plotly probability bar chart, quality table. Includes clickable sample leaves.
- **Tab 2 – Batch quality inspection:** multi-file upload → DataFrame with file, prediction, confidence %, severity %, review flag.
- **Tab 3 – Model governance:** explicit statement that metrics are synthetic, with production requirements.
- Runs on 0.0.0.0:7860; the README front-matter makes it deployable as a Hugging Face Space.

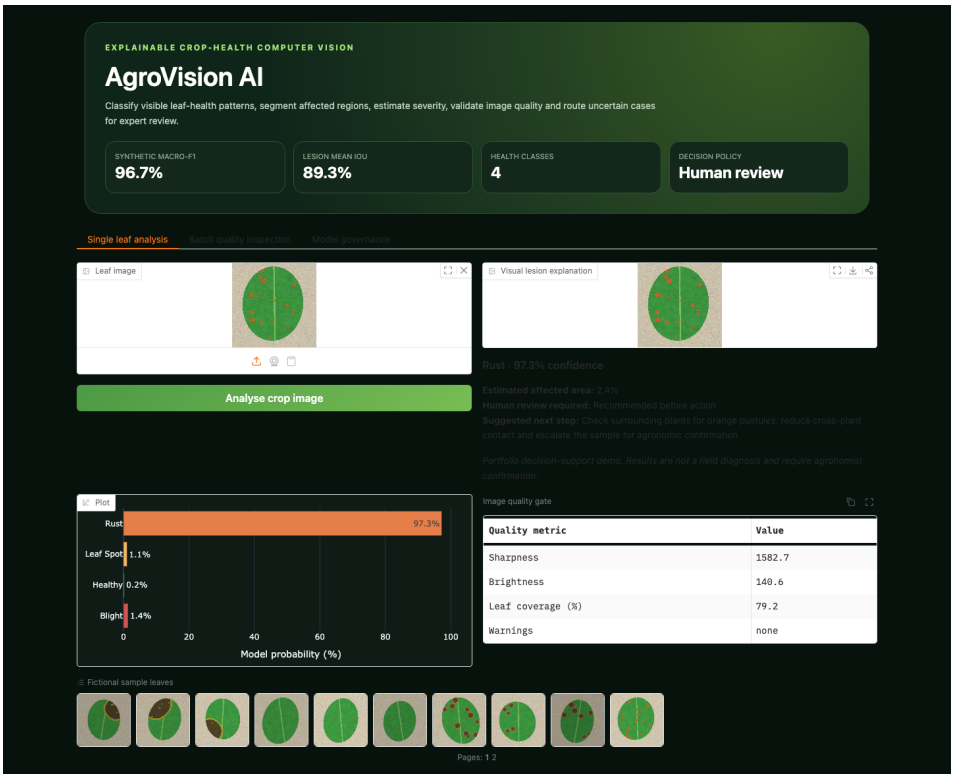



Figure 10. Gradio single-leaf tab. Note “Leaf coverage (%) 79.2” in the quality table — the bug fingerprint.

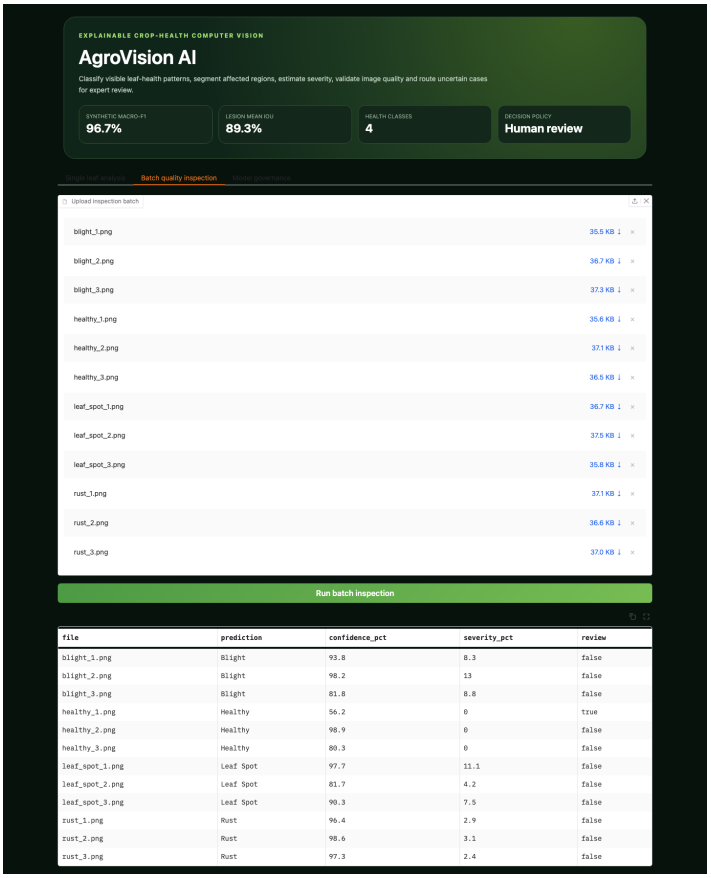



Figure 11. Batch inspection over the 12 sample images. healthy_1.png gets 56.2% confidence and review=true — it was a training image whose label was deliberately flipped to Leaf Spot.

## 15.2 FastAPI (api.py)

```
app = FastAPI(title="AgroVision AI API", version="1.0.0")
service = AgroVisionService()                    # model loaded ONCE at start-up

@app.get("/health")                              # liveness + model version
def health(): return {"status": "healthy", "model_trained_at": service.bundle.trained_at}

@app.post("/analyse")
async def analyse(file: UploadFile = File(...)):
    if not (file.content_type or "").startswith("image/"):
        raise HTTPException(415, "Upload an image file")          # Unsupported Media Type
    image = Image.open(BytesIO(await file.read())).convert("RGB")
    result = service.analyse(image); result.pop("overlay")       # PIL image isn't JSON
    return result                                                # errors → HTTP 400
```

```
$ uvicorn api:app --reload
$ curl -F "file=@data/samples/rust_1.png" http://127.0.0.1:8000/analyse
$ open http://127.0.0.1:8000/docs        # auto-generated Swagger UI
```

- Load the model once at start-up, not per request (≈80 ms per call on CPU).
- Improvements: return the overlay as base64 or a URL; file-size limits; batch endpoint; API keys/auth; request logging for monitoring; `analyse()` is CPU-bound inside an async route — run it in a threadpool (a plain `def` route) or behind workers.

## 15.3 Docker, Makefile, CI

```
FROM python:3.12-slim
ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1 PIP_NO_CACHE_DIR=1
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt     # cached layer: deps change rarely
COPY . .
EXPOSE 7860
CMD ["python", "app.py"]
```

| Make target | Runs |
|---|---|
| make install | pip install -r requirements.txt |
| make train | python train.py |
| make test | unit tests + compileall (syntax check) |
| make demo | python app.py (Gradio) |
| make api | uvicorn api:app --reload |
| make preview | regenerate the demo poster |

**CI (GitHub Actions):** on every push and pull request → Ubuntu, Python 3.12, pip cache → install → run unit tests → compile all modules.

## 15.4 Unit tests (tests/test_core.py)

| Test | Checks | Type |
|---|---|---|
| test_generator_contract | image is 128×128×3; leaf > 1000 px; lesion > 10 px | data contract |
| test_feature_contract | feature vector length == len(FEATURE_NAMES) (42) and all finite | schema / NaN guard |
| test_lesion_segmentation_overlap | lesion IoU > 0.55 on a rust leaf | algorithm quality |
| test_service_analysis | end-to-end analyse(): valid class, overlay 128×128, severity ≥ 0 | integration |
| test_saved_model_quality | saved model macro-F1 > 0.85 | regression gate on model quality |

**Missing test that would have caught the bug:** leaf-mask IoU against the generator's true leaf mask (e.g. assert IoU > 0.85).

# 16. Deep-learning path — MobileNetV3 transfer learning

`training/train_transfer_learning.py` is the bridge to real data. It expects an ImageFolder layout: `real_data/train/<class>/*.jpg` and `real_data/val/<class>/*.jpg`.

## 16.1 Concepts

- **Transfer learning:** start from a network pre-trained on ImageNet (1.28 M images). Its early layers already detect edges, colours and textures — you only adapt it to your classes, so a few thousand labelled images are enough.
- **MobileNetV3-Small:** ~2.5 M parameters, designed for phones. Uses **depthwise-separable convolutions** (a per-channel spatial conv + a 1×1 conv: ~8–9× fewer operations), **squeeze-and-excitation** blocks (channel attention) and the **hard-swish** activation; architecture found by neural architecture search.
- **Fine-tuning strategy here:** replace the final layer `classifier[3]` with a Linear layer of N outputs and train *all* layers with a small learning rate (2e-4).

## 16.2 Code walk-through

```
train tf: Resize(224) → RandomHorizontalFlip → RandomRotation(18°) → ColorJitter(.2,.2,.2)
          → ToTensor → Normalize(ImageNet mean [.485,.456,.406], std [.229,.224,.225])
val tf:   Resize(224) → ToTensor → Normalize   (no augmentation at evaluation)

model = mobilenet_v3_small(weights=DEFAULT)                    # ImageNet weights
model.classifier[3] = nn.Linear(in_features, n_classes)        # new head
optimiser = AdamW(lr=2e-4, weight_decay=1e-4); criterion = CrossEntropyLoss()
for epoch in range(epochs):
    model.train(): for batch → zero_grad → forward → loss → backward → step
    model.eval(); with torch.no_grad(): accuracy on val
    if accuracy improves: save state_dict → artifacts/deep_model/mobilenetv3_best.pt
save metadata.json (class names, best accuracy)
```

| Element | Why |
|---|---|
| Resize 224 | input size the ImageNet weights were trained on |
| ImageNet normalisation | inputs must match the statistics the pre-trained weights expect |
| Flip / rotation / colour jitter | augmentation: simulates orientation and lighting variation → better generalisation |
| AdamW | Adam with decoupled weight decay (better regularisation than L2 in Adam) |
| Cross-entropy | standard multiclass loss; −log(probability of the true class) |
| model.eval() + no_grad() | disables dropout/batch-norm updates and gradient tracking during validation |
| Save best checkpoint | simple early-stopping-style model selection |

**Be honest about its status**

The script has **not been run** (no artefacts) and is **not wired into the app**. Gaps to mention and fix:

• It selects the best epoch on the validation set and reports that same accuracy → optimistic; keep a separate test set. • Reports accuracy only → add macro-F1, per-class recall, confusion matrix. • No class weighting / weighted sampler for imbalance. • No LR schedule, no freezing-then-unfreezing, no mixed precision. • Resize(224,224) distorts aspect ratio → RandomResizedCrop is more standard.

**Candidate real datasets: PlantVillage** (~54k images, 38 crop–disease classes; lab photos on plain backgrounds — models are known to learn background shortcuts and drop sharply on field photos) and **PlantDoc** (in-the-wild field images, much harder). Train on one, test on the other to measure domain shift.

# 17. Critical review — what I found when re-running the code

Every claim below was verified by executing the project's own code. Ranked by importance.

| # | Finding | Evidence | Fix |
|---|---|---|---|
| 1 | **Leaf segmentation returns a fixed square** for every image | coverage always 79.2%; leaf-mask IoU 0.54; leaf_area_pixels ≈ 114² | excess-green + convex hull (IoU → ~0.95); add a leaf-IoU unit test |
| 2 | **Reported 0.967 macro-F1 = injected label noise** | 25 labels flipped; all 6 val errors are flipped labels; vs. true labels F1 = 1.000 | report clean-label metrics separately; the synthetic task is solved — real data is the challenge |
| 3 | Severity under-estimated ~2× | Leaf Spot 8.2% vs true 15.0%; Blight 11.4% vs 26.4% | follows from fixing #1; report severity MAE |
| 4 | Segmentation IoU is circular | thresholds tuned on the same generator that gives ground truth | evaluate on expert-annotated real masks |
| 5 | Permutation importance ≈ 0 for 40/42 features | 41 feature pairs with \|r\| > 0.9 | grouped permutation / SHAP; prune redundant features |
| 6 | No out-of-distribution rejection | random noise → Blight 55%; leaf on green background → Healthy 83%, no review | OOD detector / “not a leaf” class / energy score |
| 7 | Uncalibrated confidence | RF vote fractions used as probabilities | CalibratedClassifierCV + reliability diagram |
| 8 | UI sample images are training images | healthy_1.png is a flipped training sample | show held-out samples in the demo |
| 9 | Aspect-ratio distortion; blur measured after downscale | resize((128,128)) | letterbox; compute quality on full resolution |
| 10 | Deep-learning script unrun, unintegrated, no test set | no artifacts/deep_model | run on PlantVillage/PlantDoc; plug in behind the same service interface |
| 11 | joblib/pickle model portability | version-mismatch warning on load; pickle can execute code | pin versions; skops or ONNX for safe, portable models |

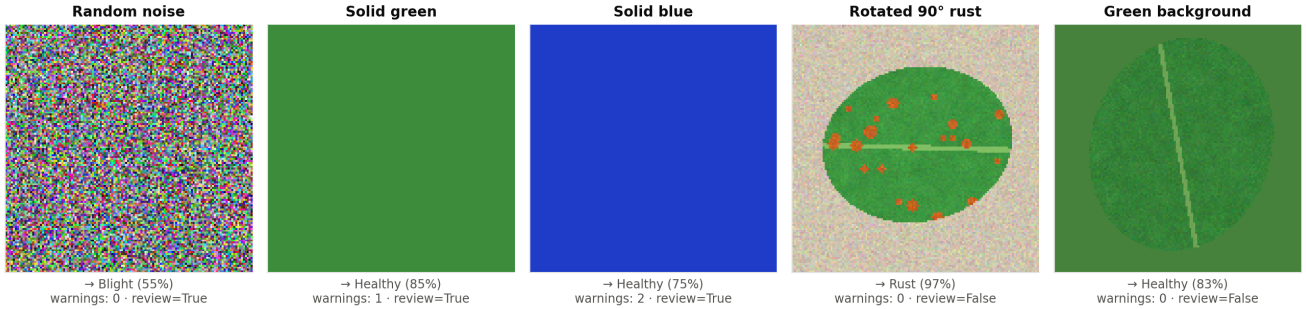



Figure 12. Out-of-distribution inputs. The system always outputs one of the four classes. Review is triggered by luck of other signals (low confidence, blur warning) — not because it recognises the input isn't a leaf. A real leaf on a green background is confidently called Healthy with no review.

**How to present the label-noise finding**

“The synthetic task is essentially solved: on the generator's clean labels the classifier makes zero errors. I deliberately injected ~3.5% label noise to mimic real annotation mistakes, and the 6 validation ‘errors’ are exactly those flipped labels — the model confidently predicted the true class. So 96.7% shows the pipeline works end to end, not field performance. The real work is collecting expert-labelled field data and validating with grouped splits.”

# 18. From demo to production — roadmap

| Area | What to do |
|---|---|
| Data | collect field photos across crops, regions, seasons, cameras; agronomist labels with inter-annotator agreement (Cohen's κ); add 'other / unknown' and look-alike classes (nutrient deficiency, pests) |
| Splits | grouped by farm/region/season; separate untouched test set; also a time-based split to test future performance |
| Classification | fine-tune MobileNetV3 / EfficientNet / ConvNeXt / ViT (DINOv2 features + linear head is a strong baseline); class-balanced loss or focal loss |
| Segmentation | U-Net / DeepLabV3+ trained on expert masks, or SAM2 prompted/fine-tuned; report IoU, Dice, severity MAE |
| Uncertainty | temperature scaling / isotonic calibration; ensembles or MC-dropout; OOD detection (energy score, Mahalanobis, dedicated 'not a leaf' classifier) |
| Edge deployment | export to ONNX / TFLite / Core ML; INT8 quantisation; on-device quality gate so farmers can retake photos offline |
| MLOps | experiment tracking (MLflow), model registry and versioning, data versioning (DVC), CI/CD with model-quality gates, canary rollout |
| Monitoring | input drift (colour/brightness/blur distributions, embedding drift), prediction-mix drift, review-rate, expert-override rate, latency; periodic retraining from reviewed cases (active learning) |
| Governance | model card (exists), data sheet, audit logs of every decision, clear 'decision support, not diagnosis' UX |

## 18.1 Monitoring in one paragraph

After deployment the world changes: new phones, a new region, a new season, a new disease strain. Track (1) **data drift** — distributions of brightness, sharpness, colour histograms or CNN embeddings vs. training (PSI, KS test); (2) **prediction drift** — the class mix and confidence distribution; (3) **outcome quality** — expert-override rate on reviewed cases, which is your delayed ground truth. Trigger retraining when these cross thresholds, and feed reviewed images back into the training set.

# 19. Interview questions and model answers

## Project overview

**Q: Walk me through your project.**

Use the 30-second pitch (§1.2), then the pipeline diagram: quality gate → leaf mask → lesion mask and severity → 42 features → Random Forest → review routing → Gradio/FastAPI. Finish with the honest caveat and the real-data plan.

**Q: What was the hardest part?**

Getting trustworthy evaluation without real data — building a generator that gives ground-truth masks so segmentation could be measured, and then auditing my own metrics (finding the label-noise effect and the leaf-mask bug).

**Q: What would you do differently?**

Unit-test intermediate outputs (leaf-mask IoU), report clean-label metrics separately, use SHAP instead of permutation importance, and start the real-data CNN path earlier.

**Q: Why synthetic data?**

Reproducibility, zero download, and exact pixel masks for IoU/Dice. The trade-off is no field validity, so I designed it as a pipeline ready to swap in real data.

## Computer vision

**Q: How does your leaf segmentation work, and does it work?**

Green-dominance threshold → dilation → hole filling → erosion → opening → largest component. When auditing I found the beige background also passes the threshold, so it produces a fixed square. An excess-green index with a convex hull fixes it (IoU 0.54 → ~0.95).

**Q: Explain dilation, erosion, opening, closing.**

Dilation grows white regions; erosion shrinks them. Opening = erosion then dilation (removes small specks). Closing = dilation then erosion (fills small holes).

**Q: How do you detect blur?**

Variance of the Laplacian. The Laplacian is a second-derivative filter that responds to edges; a sharp image has many strong responses so the variance is high; blur flattens them.

**Q: Why RGB thresholds? What breaks on real photos?**

They're transparent and need no training, but they're sensitive to lighting, white balance and camera. HSV/Lab colour spaces help a bit; a learned segmentation model is the real fix.

**Q: IoU vs Dice?**

Both measure overlap. IoU = intersection/union; Dice = 2·intersection/(sum of sizes). Dice = 2·IoU/(1+IoU), so it's always ≥ IoU and they rank models identically.

**Q: Why is rust IoU lower than leaf spot?**

Rust pustules are tiny; a one-pixel boundary error is a large fraction of a 2–3 px object. Small objects are always harder for overlap metrics.

## Machine learning

**Q: Why a Random Forest?**

Small tabular feature set (42 features, 720 rows): RF is robust, needs no scaling, trains in ~1 s on CPU, gives probabilities and importances. For real photos I'd switch to a CNN.

**Q: How does a Random Forest work?**

Many decision trees, each trained on a bootstrap sample and considering a random subset of features at each split; predictions are averaged. Bagging + feature randomness reduces variance.

**Q: What does class_weight='balanced_subsample' do?**

Weights each class inversely to its frequency, recomputed inside each tree's bootstrap sample — so minority classes aren't ignored.

**Q: How did you tune hyperparameters?**

Honestly, they were set manually. I'd use randomised search with stratified or grouped cross-validation, scored on macro-F1.

**Q: Your model gets 96.7%. Is it overfitting?**

No — it's 100% on the clean labels; the 3.3% gap is injected label noise. The real risk isn't overfitting, it's that the synthetic task is too easy and won't transfer to real data (domain shift).

**Q: How would you handle class imbalance in real data?**

Stratified/grouped splits, class weights or a weighted sampler, focal loss for the CNN, macro-F1 and per-class recall as metrics, per-class thresholds.

**Q: Why are most permutation importances zero?**

Correlated features: shuffle one and the forest uses its correlated twin. Use grouped permutation or SHAP.

**Q: What is label noise and how would you handle it?**

Wrong labels from annotators. Detect with confident learning (cleanlab), multiple annotators and agreement (κ), relabel disputed cases; use robust losses (label smoothing).

## Metrics & validation

**Q: Why macro-F1 instead of accuracy?**

Accuracy is dominated by the majority class (mostly-healthy fields). Macro-F1 weights each class equally, so a model that misses all disease scores badly.

**Q: Precision or recall — which matters more here?**

Recall for disease classes: a missed blight is worse than an extra expert check. I'd tune per-class thresholds to hit a recall target, then minimise review workload.

**Q: How would you validate on real data?**

Grouped splits by farm/region/season to avoid leakage, an untouched test set, per-class recall/precision, macro-F1, confusion matrix, calibration (ECE), IoU/Dice on expert masks, severity MAE, and error analysis by subgroup (crop, camera, lighting).

**Q: What is data leakage? Give an example here.**

Information from the test set leaking into training. Here: photos of the same plant or field in both train and test — the model memorises the background/plant, not the disease.

**Q: What is calibration?**

Whether a predicted 0.8 is right 80% of the time. Check with a reliability diagram/ECE; fix with temperature scaling, Platt or isotonic regression.

## Deep learning

**Q: What is transfer learning and why use it?**

Reuse a network pre-trained on ImageNet; early layers already detect generic features. You fine-tune on a few thousand images instead of millions.

**Q: Why MobileNetV3?**

Small and fast (depthwise-separable convolutions, SE blocks, hard-swish) — suitable for phones in fields with poor connectivity.

**Q: Why ImageNet normalisation?**

Pre-trained weights expect inputs with the same mean/std they were trained on.

**Q: Feature extraction vs fine-tuning?**

Feature extraction freezes the backbone and trains only the head (good with very little data). Fine-tuning updates all layers with a small LR (better with more data). Common: freeze first, then unfreeze.

**Q: How would you explain a CNN's prediction?**

Grad-CAM heatmaps, compared with the lesion overlay; check it focuses on lesions, not background (shortcut learning).

## Engineering & MLOps

**Q: Why both Gradio and FastAPI?**

Gradio is for humans (demo, inspection); FastAPI is for machines (integration with mobile apps or dashboards). Both call the same service layer.

**Q: How is the model versioned?**

The bundle stores metrics and a trained_at timestamp, exposed via /health. In production: a model registry (MLflow) with data and code versions.

**Q: How would you scale the API?**

Load the model once; run CPU-bound inference in worker threads/processes; multiple Uvicorn/Gunicorn workers behind a load balancer; batch requests; or ONNX Runtime for speed.

**Q: What do your tests cover?**

Data contract, feature schema, segmentation quality, end-to-end service, model-quality regression gate — run in GitHub Actions on every push.

**Q: How would you monitor it after deployment?**

Data drift (image statistics/embeddings), prediction drift, review and expert-override rates, latency; retrain from reviewed cases.

## Business & ethics

**Q: How does this help a business?**

Prioritises expert time, standardises inspection, quantifies severity for tracking spread, and creates an auditable review queue.

**Q: What could go wrong ethically?**

Over-trust leading to wrong pesticide use or missed outbreaks; bias against crops/regions not in the training data. Mitigate with human review, clear disclaimers, subgroup evaluation and OOD rejection.

# 20. Glossary

| Term | Definition |
|---|---|
| **Bagging** | training models on bootstrap samples and averaging them |
| **Bootstrap sample** | random sample with replacement, same size as the data |
| **Calibration** | agreement between predicted probabilities and observed frequencies |
| **Confusion matrix** | table of actual vs predicted classes |
| **Connected component** | a group of touching pixels with the same value |
| **Data drift** | change in input distribution after deployment |
| **Dice coefficient** | $2\|A \cap B\|/(\|A\|+\|B\|)$; pixel-level F1 |
| **Domain shift** | difference between training data and deployment data (e.g. synthetic → field) |
| **ECE** | Expected Calibration Error — average gap between confidence and accuracy across bins |
| **Excess-green index** | $2G - R - B$; standard vegetation segmentation index |
| **Gini impurity** | $1 - \Sigma\, p_k^2$; how mixed a tree node is |
| **Grouped split** | split where all samples of a group (farm, plant) go to the same side |
| **Human-in-the-loop** | humans review uncertain model outputs; their decisions improve the model |
| **IoU / Jaccard** | intersection over union of two masks |
| **Label noise** | incorrect labels in the training/validation data |
| **Laplacian** | second-derivative filter; highlights edges; its variance measures sharpness |
| **Macro-F1** | unweighted mean of per-class F1 |
| **MDI importance** | mean decrease in impurity — RF's built-in feature importance |
| **Morphology** | shape operations on binary masks: dilation, erosion, opening, closing |
| **OOD** | out-of-distribution input, unlike anything seen in training |
| **Permutation importance** | score drop when a feature is randomly shuffled |
| **Procedural generation** | creating data algorithmically from rules and random seeds |
| **SHAP** | game-theory-based per-prediction feature attributions |
| **Sobel filter** | first-derivative filter for edge/gradient detection |
| **Stratified split** | split preserving class proportions |
| **Transfer learning** | re-using a pre-trained network for a new task |

# 21. One-page cheat sheet

| Numbers to remember | |
|---|---|
| Classes | 4 — Healthy, Leaf Spot, Rust, Blight |
| Images | 720 = 4 × 180; 540 train / 180 val; 128×128 RGB |
| Features | 42 = 3 mean + 3 std + 24 histogram + 3 gradient + 5 ratios + 4 quadrants |
| Model | Random Forest, 420 trees, depth 16, min leaf 2, balanced_subsample, seed 42 |
| Reported | macro-F1 0.967, accuracy 0.967, IoU 0.893, Dice 0.941 |
| Reality | 25 labels (3.5%) flipped; all 6 val errors are flipped labels; clean-label F1 = 1.000 |
| Per-class F1 | Healthy 0.956, Leaf Spot 0.966, Rust 0.978, Blight 0.967 |
| Lesion IoU by class | Leaf Spot 0.99, Rust 0.88, Blight 0.80 |
| Quality thresholds | blur < 65 (Laplacian var), dark < 55, bright > 220, coverage < 12% |
| Review rule | confidence < 0.65 OR any warning |
| Leaf-mask bug | always 114×114 square = 79.2% coverage; leaf IoU 0.54 → ~0.95 with fix |
| Deep path | MobileNetV3-Small, 224 px, ImageNet norm, AdamW 2e-4, wd 1e-4, CE loss, 8 epochs, batch 32 |
| Latency | ≈82 ms per image (CPU), RF training ≈1 s |

| Formulas | |
|---|---|
| Precision / Recall | TP/(TP+FP) / TP/(TP+FN) |
| F1 | 2PR/(P+R) |
| Macro-F1 | mean of per-class F1 |
| IoU | \|A∩B\| / \|A∪B\| |
| Dice | 2\|A∩B\| / (\|A\|+\|B\|) = 2·IoU/(1+IoU) |
| Severity | lesion pixels / leaf pixels × 100 |
| Gini | $1 - \Sigma\ p_k^2$ |
| Excess green | ExG = 2G − R − B |
| Sharpness | Var(Laplacian(grey)) |

**Three sentences to close any answer about this project**

“The pipeline is complete and production-shaped: quality gate, classification, segmentation, severity, human review, UI, API, Docker, tests and CI. The metrics are synthetic and I audited them myself — I know exactly where they overstate performance. With real expert-labelled data and grouped validation, the same architecture becomes a genuine field triage tool.”

# 22. How to run everything

```
# setup
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt

python train.py                       # train + write artifacts/metrics.json
python app.py                         # Gradio UI → http://127.0.0.1:7860
uvicorn api:app --reload              # API → http://127.0.0.1:8000/docs
python -m unittest discover -s tests -v
python create_demo_preview.py         # regenerate the poster image

# docker
docker build -t agrovision . && docker run -p 7860:7860 agrovision

# optional deep-learning path (needs real data in ImageFolder layout)
pip install -r requirements-deep-learning.txt
python training/train_transfer_learning.py --data real_data --epochs 8
```

**Before the interview**

1. Run `python app.py` once and click through all three tabs, so you can describe the UI from experience.
2. Be able to draw Figure 1 from memory.
3. Say the 30-second pitch and the label-noise answer out loud twice.

Good luck!